

E3-TP3 : LANGAGES COMPILÉS

Prérequis : E3-Fiche3 : Étapes de compilation	Matériels et documentation : - LMDE5 - Build Essential	Durée : 1H30
--	--	-----------------------------------

Problématique	Comment compiler un projet écrit en C++ ?
Objectifs	À la fin de la séance, vous devrez être capable de : - Compiler un projet C++ avec g++ ou make - Écrire un fichier Makefile pour simplifier la compilation

On vous demande de mettre en place un fichier **Makefile** permettant d'automatiser la compilation de code C++.

1 AVANT-PROPOS : LES LANGAGES COMPILÉS

Un **langage compilé** (C, C++, Rust, Fortran, Ada, Swift...) est un langage de programmation dont le code source est traduit en langage machine ou en code intermédiaire par un compilateur avant l'exécution. Contrairement aux langages interprétés, le code source n'est pas directement exécuté, mais plutôt transformé en un format compréhensible par la machine.

Processus de compilation C/C++	
Pré-traitement (processeur) / Compilation	Édition de Liens

Performance + : Le code compilé est généralement plus rapide à l'**exécution** que le code interprété.

Optimisations + : Les compilateurs effectuent des optimisations pour améliorer l'efficacité du code.

Protection du code source + : Le code source n'est pas distribué, renforçant la sécurité et protégeant la propriété intellectuelle.

Temps de compilation - : Le processus de compilation peut prendre du temps, surtout pour de grands projets.

Portabilité limitée - : Le code compilé peut être spécifique à une architecture, nécessitant parfois des ajustements pour chaque plateforme cible.

Utilisations : Systèmes embarqués, développement logiciel, développement système, applications nécessitant des performances élevées (jeux vidéo, calcul scientifique).

Une grande partie du code des applications suivantes est écrite en **C++** : Microsoft Office, Adobe Photoshop, Autodesk AutoCAD, Mozilla Firefox, Google Chrome... Des moteurs de jeux tels que Unreal Engine et Unity Engine utilisent le C++ pour le développement de jeux.

Les langages compilés offrent des performances optimales et une sécurité accrue, bien que le processus de compilation puisse être plus complexe. Ils sont privilégiés pour des applications critiques en termes de performance et de sécurité, mais peuvent nécessiter une gestion plus approfondie de la portabilité.

2 ACTIVITÉ 1 : COMPILEUR DIRECTE

- (01) Rappeler le nom du compilateur C++ utilisé par votre distribution (voir E3-TP1).
- (02) Rappeler les deux étapes d'une compilation permettant d'obtenir un binaire à partir de sources C++.

1.1 COMPILEUR DIRECTE D'UNE APPLICATION CONSOLE (1 FICHIER)

- (03) Créer un fichier source **main_act_1.cpp** dans un répertoire nommé **tp3/act1**. Éditer le fichier avec Sublime-Text et écrire le code C++ suivant :

```
#include <iostream>
using namespace std;

int main() {
    cout << "Activité 1" << endl;
    cout << "Code C++ compilé avec g++" << endl;
    return 0;
}
```



- (04) Compiler directement le fichier **main_act_1.cpp** en effectuant la commande `$ g++ main_act_1.cpp`. Rechercher et exécuter le fichier binaire généré.
- (05) Recompiler le source **main_act_1.cpp** en ajoutant une option permettant de définir le nom du fichier exécutable. Donner la commande et l'exécuter. Exécuter le binaire obtenu.

1.2 VARIABLE D'ENVIRONNEMENT « ? »

Si l'exécution du programme se déroule correctement, il doit renvoyer la valeur 0 au système (`return 0;`), cette valeur est stockée automatiquement dans la variable d'environnement nommée « ? ».

- (06) Rappeler la commande permettant d'afficher la valeur de la variable d'environnement « ? ». Exécuter cette commande et vérifier sa valeur.

2 ACTIVITÉ 2 : COMPILEUR SÉPARÉE

2.1 COMPILEUR SÉPARÉE D'UNE APPLICATION CONSOLE (PLUSIEURS FICHIERS SOURCES)

- (07) Créer les trois fichiers suivants dans le répertoire **tp3/act2** :

act2/main.cpp	act2/function.h	act2/function.cpp
<pre>#include "function.h" int main() { display(); return 0; }</pre>	<pre>#ifndef FUNCTION_H #define FUNCTION_H #include <iostream> void display(); #endif</pre>	<pre>#include "function.h" void display() { std::cout << "Activité 2\n"; }</pre>



- (08) Donner la commande permettant de compiler un fichier (.cpp) pour obtenir un fichier objet (.o). Compiler les deux fichiers (.cpp) à l'aide de cette commande.
- (09) Donner la commande permettant de réaliser l'édition de lien des deux fichiers objets (.o) obtenus précédemment. Réaliser l'édition de lien avec cette commande et exécuter le fichier binaire obtenu.

2.1 COMPILATION SÉPARÉE EN UTILISANT MAKE

Nous nous rendons vite compte que la solution de compilation précédente devient longue si le projet est composé de beaucoup de fichiers sources. Pour automatiser le processus, la solution est d'utiliser l'outil **make**. Cependant il ne peut pas être utilisé ici directement, il faut lui définir un fichier **Makefile** où nous définirons toutes les règles de compilations ainsi que les dépendances. Le fichier **Makefile** est appelé implicitement par la commande **make**.

- **Makefile de base :**

- (10) Saisir le code suivant et le sauvegarder sous le nom **Makefile** (attention à la tabulation obligatoire aux deux dernières lignes). Exécuter la commande **make** et tester le résultat.

```
# Makefile
all: hello
hello: main.cpp function.cpp
    g++ -O2 -Wall -g -c main.cpp function.cpp
    g++ -o hello main.o function.o
```



- **Makefile portable :**

Pour améliorer la portabilité du fichier **Makefile**, on préférera utiliser des variables. Ainsi le **Makefile** pourra être adapté à de nombreux programmes.

```
# Nom du programme
PROG = ...
# Liste des fichiers sources (.cpp) séparés d'un espace
SRCS = ...
# Conversion automatique des fichiers sources (.cpp) en fichiers objets (.o)
OBJECTS = $(SRCS:.cpp=.o)
# Options préprocesseur,
CPPFLAGS =
# Options compilateur,
CXXFLAGS = -O2 -Wall -g
# Options éditeur de lien
LDFLAGS =

# Compilation
all: $(PROG)
$(PROG): $(SRCS)
    $(CXX) $(CPPFLAGS) $(CXXFLAGS) -c $(SRCS)
    $(CXX) $(LDFLAGS) -o $(PROG) $(OBJECTS)
```



- (11) Compléter le **Makefile** et exécuter la commande **\$ make**. Expliquer les 2 dernières lignes du fichier **Makefile**.
- (12) Modifier le fichier **Makefile** en ajoutant l'option de compilation pour utiliser la norme 2011 de C++.

- **Faire le nettoyage :**

Pour supprimer tous les fichiers compilés (fichiers objets), nous souhaitons exécuter la commande : **\$ make clean**.

- (13) Modifier le fichier **Makefile** pour faire le nettoyage en utilisant la commande **\$ make clean**.